

Chain of Thought: *o raciocínio à mostra*

Na técnica **Chain of Thought** (CoT, cadeia de pensamento), você pede que o modelo **mostre o raciocínio** antes da conclusão. Na aula 10, quem definia os passos era você; aqui, o próprio modelo monta o caminho. Nesta aula, com o notebook `08-cot.ipynb`: CoT com tags `<cot>` em marketing, saúde, investimentos e produto, o CoT simples e a diferença para o raciocínio interno do gpt-oss.

`<cot> ... </cot>`

Papel developer

Auditar o raciocínio

Passo a passo

`message.reasoning`

Formação Claude Code 2026 · Baseado em console.groq.com/docs/reasoning, console.groq.com/docs/api-reference e no Guia de Engenharia de Prompt da aula 16 (seção 8), consultados em outubro de 2026 · Código e saídas reais gravadas no notebook `08-cot.ipynb` (modelo `openai/gpt-oss-120b`)

Pensar antes, *concluir depois*

08-cot.ipynb · célula de texto

Útil em situações:

- Raciocínio estruturado e explícito
- Explicação transparente do raciocínio
- Clareza na tomada de decisão

<COT>

hipóteses, contas,
critérios



</COT>

fim do raciocínio



CONCLUSÃO

objetiva

- ✓ O padrão do notebook: "Mostre o raciocínio dentro de **<cot>** e depois dê uma conclusão objetiva"
- ✓ As tags separam o **caminho** do **resultado**: dá para exibir só a conclusão ao usuário e guardar o raciocínio para revisão
- ✓ O grande ganho é a **auditoria**: se a conclusão estiver errada, você vê em qual passo o erro entrou

Aula 10 × aula 11: em múltiplos passos, você escreve a metodologia; no CoT, você pede que o modelo exponha a dele.

CoT numa *estratégia de marketing*

célula 5

```
system_prompt = """
Você é um estrategista de marketing que sempre usa Chain of Thoughts (CoT).
Mostre o raciocínio dentro de <cot> e depois dê uma conclusão objetiva.
"""

user_prompt = """
Uma empresa de moda quer lançar uma campanha para atrair a Geração Z.
Dados:
- Orçamento: R$ 500 mil
- Principais canais: Instagram, TikTok, YouTube
- Objetivo: aumentar vendas online em 30% em 6 meses
- Concorrência: 3 marcas fortes no mesmo segmento

Crie um plano de campanha explicando o raciocínio passo a passo.
"""

response = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[{"role": "developer", "content": system_prompt},
              {"role": "user", "content": user_prompt}],
    temperature=0.4
)
...
```

SAÍDA GRAVADA (INÍCIO)

```
Plano de Marketing: <cot>
**1. Entender o público-alvo (Gen Z)**
- Idade: 16-24 anos.
- Valores: autenticidade, diversidade, sustentabilidade,
rapidez e conteúdo visual/curto.
- Hábitos: consumo intenso de vídeo curto (TikTok, Reels),
busca por tendências em tempo real, alta confiança em
influenciadores micro-e nano-influenciadores.

**2. Analisar o cenário competitivo**
- 3 concorrentes fortes já investem pesado em TikTok e
colaborações com creators.
- Diferencial necessário: storytelling de marca + foco em
sustentabilidade + experiências interativas (AR,
challenges).

**3. Definir metas SMART**
- Aumentar vendas online 30% em 6 meses → crescimento médio
mensal de ≈4,5% nas conversões.
...
```

- ✓ Repare em `"role": "developer"`: o SDK do Groq aceita `developer` como sinônimo de `system`
- ✓ Conferido: crescer 4,5% ao mês por 6 meses dá $1,045^6 \approx 1,30$, ou seja, +30%

Hipóteses clínicas: *o que o modelo supôs*

célula 7 (prompts)

```
system_prompt = """
Você é um médico que usa Chain of Thoughts (CoT) para raciocínio clínico.
Sempre apresente hipóteses dentro de <cot> e finalize com recomendação
objetiva.
"""

user_prompt = """
Paciente: 45 anos, sedentário, fumante, apresenta dores no peito durante
esforço físico.
Exames iniciais: colesterol alto, pressão arterial elevada.

Qual deve ser a recomendação inicial para o paciente?
"""
...
```

Exemplo didático de técnica de prompt, **não** orientação médica. A resposta chega a sugerir remédios e doses: decisão clínica é de profissional de saúde.

SAÍDA GRAVADA (INÍCIO)

```
Análise clínica: <cot>
- **Hipótese principal:** Angina de esforço devido a doença
arterial coronariana (DAC) incipiente, considerando dor
torácica associada ao esforço, hipertensão e
hipercolesterolemia.
- **Hipóteses secundárias:**
  1. Dor torácica musculoesquelética relacionada ao
  sedentarismo.
  2. Doença pulmonar obstrutiva crônica (DPOC) precoce por
  tabagismo, com dor torácica referida.
  3. Síndrome metabólica (obesidade, hipertensão,
  dislipidemia, tabagismo) predispondo a eventos
  cardiovasculares.
- **Fatores de risco relevantes:** idade (45 anos),
  tabagismo ativo, sedentarismo, colesterol LDL elevado,
  pressão arterial sistólica > 140 mmHg.
...
```

- ✓ O CoT mostra o que o modelo **acrescentou**: "obesidade" e "PA sistólica > 140 mmHg" não estavam nos dados do paciente
- ✓ Sem o raciocínio exposto, essas suposições passariam despercebidas

Auditando *as contas* do raciocínio

CÉLULA 9 · TRECHOS DA SAÍDA GRAVADA

- Receita atual: R\$ 300 mil/mês → R\$ 3,6 mi/ano.
- Crescimento de 12% ao mês equivale a aproximadamente 3,8 × ao ano ($(1,12)^{12} \approx 3,8$).
- Projeção de receita anual em 12 meses: R\$ 3,6 mi × 3,8 ≈ R\$ 13,7 mi.
- Receita projetada em 24 meses: R\$ 13,7 mi × 3,8 ≈ R\$ 52 mi.

...

- Mesmo com custos totais de ~R\$ 2,5 mi/ano, a margem EBITDA projetada para o próximo ano seria: (R\$ 13,7 mi – R\$ 2,5 mi) ≈ R\$ 11,2 mi → margem EBITDA ≈ 82%. Essa margem parece ****excessivamente alta****, indicando que os custos provavelmente estão subestimados ou que a projeção de receita é otimista.

...

Prompt: startup com receita de R\$ 300 mil/mês, crescendo 12% ao mês, levantando R\$ 5 milhões.

NO RACIOCÍNIO	CONFERÊNCIA
$1,12^{12} \approx 3,8$	$1,12^{12} = \mathbf{3,90}$: arredondou para baixo
"Receita anual em 12 meses ≈ R\$ 13,7 mi"	É o ritmo anual no 12º mês. A receita somada dos próximos 12 meses dá ≈ R\$ 8,1 mi
Margem EBITDA a partir de R\$ 2,5 mi de custos	≈ 82%, irreal; o próprio modelo depois sugere 30 a 40%
R\$ 5 mi por 20% → pré-money R\$ 20 mi	Correto

O CoT não garante acerto, mas deixa cada número à vista. É por isso que dá para encontrar os erros.

Priorizando o *MVP*

célula 11 (user)

```
user_prompt = """
Estamos desenvolvendo um aplicativo de bem-estar.
Funcionalidades propostas:
1. Meditação guiada
2. Monitoramento de sono
3. Gamificação de hábitos
4. Relatórios personalizados

Orçamento permite lançar apenas duas no MVP.
Qual priorizar?
"""

...
```

- ✓ O raciocínio usa um **critério explícito**: "alto impacto + esforço moderado"
- ✓ Verifica dependências: relatórios precisam de dados que meditação e sono geram

SAÍDA GRAVADA (FINAL)

```
**4. Verificar risco de dependência**
- Relatórios personalizados precisam de dados; ao escolher
meditação + sono já temos duas fontes de dados suficientes
para gerar relatórios simples no futuro, sem precisar
lançá-los no MVP.
- Gamificação pode ser adicionada depois, usando os mesmos
eventos de meditação e sono como gatilhos de recompensas.

**5. Conclusão**
As duas funcionalidades que melhor equilibram valor ao
usuário, viabilidade de entrega e potencial de diferenciação
são **Meditação guiada** e **Monitoramento de sono**. Elas
permitem validar a proposta de bem-estar, gerar engajamento
diário e coletar dados críticos para evoluções posteriores
(relatórios e gamificação).
</cot>

**Prioridade para o MVP (duas funcionalidades):**
1 Meditação guiada
2 Monitoramento de sono
```

0 </cot> fecha o raciocínio; depois vem só a decisão, pronta para mostrar a quem pediu.

"Pense passo a passo" *sem tags*

célula 12

```
# Chain of Thoughts
cot_prompt = """
Quais as vantagens de trabalhar remoto?
Use Chain of Thoughts para pensar passo a passo antes de responder.
"""

response_cot = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[{"role": "user", "content": cot_prompt}],
    temperature=0.5
)

print("=== CHAIN OF THOUGHTS ===")
print(response_cot.choices[0].message.content)
```

SAÍDA GRAVADA (INÍCIO)

```
**Raciocínio passo a passo (Chain of Thoughts)**

1. **Identificar os principais aspectos da vida profissional
que podem ser impactados por um modelo remoto.**
    - Horário e flexibilidade.
    - Deslocamento e custos associados.
    - Ambiente de trabalho e conforto.
    - Produtividade e foco.

...
```

- ✓ Funciona, mas raciocínio e resposta saem misturados num texto só (67 linhas)
- ✓ Com tags, um código simples separa as duas partes

O gpt-oss *já pensa sozinho*

- ✓ O gpt-oss-120b é um modelo de raciocínio: antes de responder, ele produz um raciocínio interno
- ✓ No Groq, esse raciocínio vem por padrão no campo `message.reasoning`, separado de `message.content`; os notebooks só imprimem o `content`
- ✓ `include_reasoning=False` tira o raciocínio da resposta;
`reasoning_effort` (`low`, `medium`, `high`) controla o quanto ele pensa

O CoT no prompt continua útil: ele produz um raciocínio **no formato que você pediu** (tags, passos, critérios), pensado para ser lido por pessoas.

Lendo o raciocínio interno (não está no notebook)

```
r = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[{"role": "user", "content": "..."}],
    reasoning_effort="high",
)
print(r.choices[0].message.reasoning) # o "rascunho"
print(r.choices[0].message.content)  # a resposta
```

Parâmetros conforme console.groq.com/docs/reasoning (seção GPT-OSS Models).

CoT ajuda **e** custa

USE QUANDO

- ✓ Há contas, critérios ou várias etapas lógicas
- ✓ A decisão precisa ser justificada (negócios, priorização)
- ✓ Você quer auditar como a conclusão foi construída

EVITE QUANDO

- ✓ A tarefa é simples (traduzir, formatar, classificar)
- ✓ Tempo e tokens importam: o raciocínio pode multiplicar o tamanho da resposta
- ✓ O usuário final não deve ver o raciocínio: separe com tags e mostre só a conclusão

Um raciocínio bem escrito **parece** confiável. Confira números, premissas e fatos acrescentados, como nas seções 03 e 04.

Separe o raciocínio *com código*

Exercício: peça um CoT com tags para decidir entre alugar e financiar um imóvel e mostre só a conclusão.

```
texto = response.choices[0].message.content
if "</cot>" in texto:
    raciocinio, conclusao = texto.split("</cot>", 1)
    print(conclusao.strip())
else:
    print(texto)  # o modelo não usou as tags
```

- ✓ Dê dados no `user`: valor do aluguel, parcela, juros, prazo
- ✓ Confira as contas do raciocínio com uma calculadora
- ✓ Compare com a mesma pergunta sem CoT
- ✓ Desafio: imprima também `message.reasoning` e compare com o conteúdo de `<cot>`

Teste o que *ficou*

1. Para que servem as tags <cot> nos prompts?

A) Deixar mais rápido

B) Separar raciocínio e conclusão

C) Trocar de modelo

D) Reduzir custos

B.

2. No SDK do Groq, o papel "developer" equivale a:

A) user

B) assistant

C) system

D) tool

C.

3. O maior benefício do CoT nesta aula foi:

A) Respostas sempre certas

B) Poder auditar premissas e contas

C) Respostas menores

D) Dispensar dados

B.

4. Onde o gpt-oss devolve o raciocínio interno?

A) message.content

B) message.reasoning

C) usage

D) Não devolve

B.

Raciocínio visível, *erros visíveis*

Pedir o raciocínio dentro de `<cot>` e a conclusão depois deixa cada premissa e cada conta à vista. Isso não torna o modelo infalível, mas torna os erros encontráveis. No gpt-oss, há ainda o raciocínio interno em `message.reasoning`, controlado por `reasoning_effort`.

`<cot>`

Auditoria

`developer = system`

`message.reasoning`

`reasoning_effort`

Próxima aula → 12 · Trabalhando com Sumarização

Fontes: console.groq.com/docs/reasoning (GPT-OSS Models, `include_reasoning`, `reasoning_effort`); console.groq.com/docs/api-reference; SDK groq 1.7.0 (mensagem de sistema com role "system" ou "developer"); Guia de Engenharia de Prompt (aula 16), seção 8, consultados em outubro de 2026. Código e saídas: notebook `cot.ipynb` desta aula; contas da seção 04 refeitas à mão.